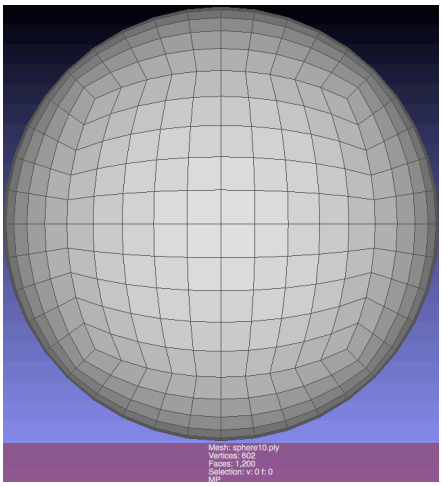
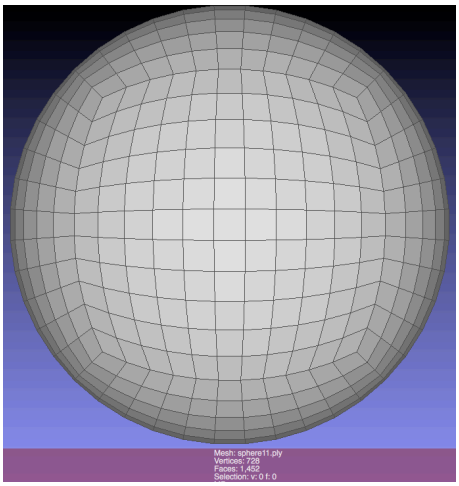
Map Quadded Unit Sphere Onto Convex Hull

Well-balanced quadded convex hulls, like their cousin the quadded unit sphere, will have vertex unit normal vectors distributed uniformly over their surfaces. This doesn't mean that the linear distance between a convex hull's adjacent vertices and normal vectors is uniform, as it is for the sphere, but only that the angular separation between adjacent normal vectors is uniform. The linear distance between adjacent normal vectors depends on the local surface curvature, which is constant on the unit sphere but may vary from 0 to infinity on the convex hull. To eliminate sharp corners and edges on a convex hull, a moderate minimum radius of curvature may be set, which will limit the maximum curvature. Linear distances between adjacent vertex normal vectors are proportional to local surface radii of curvature, so face dimensions tend to be larger on flatter surfaces and smaller on more curved surfaces.

To facilitate accessing vertices on an arbitrary convex hull, it is helpful to project vertices from a uniformly quadded unit sphere onto its surface. However, it isn't enough to just project the sphere's vertices onto the convex hull's surface because this would result in vertex normal vectors that don't align with those on the sphere. It is necessary to generate a new set of vertices on the convex hull's surface whose normal vectors align with those on the unit sphere.

Let the original convex hull be represented by a set of triangles in a .ply file as would be exported from MeshLab. If the convex hull's mesh resolution has been made sufficiently fine, its vertices can be taken as lying on or close to the desired convex hull surface. MeshLab can apply Loop subdivision any number of times to the triangulated surface to increase triangle density. Given a sufficient number of triangles, one or more vertices on the convex hull will have normal vectors within some positive ε tolerance of each normal vector on the quadded unit sphere. If no normal vector on the convex hull is within ε of some normal vector on the sphere, either ε is set too small or the surface triangles must be further subdivided.





The unit spheres in the above images have been divided into six identical major quads numbered 0 through 5. Quads 0 and 5 are centered on the sphere's respective ±z-axes, quads 1 and 3 on the ±y-axes, and quads 4 and 2 on the ±x-axes. Each major quad has been subdivided into a square array of n^2 minor quads. The resulting $6n^2+2$ vertices on the unit sphere are arranged into $3(n-1)$ counterclockwise circular rings around the sphere's positive z-axis, arranged from top (+z-axis) to bottom (–z-axis.) Each normal vector on the unit sphere is identical to the respective vertex vector, so no additional information has to be stored in the .ply file for the unit sphere.

The above figures show one major quad centered in each image, which may be taken as major quad 0 viewed from the positive z-axis. Edges of major quads 1 to 4, adjacent to quad 0, are visible in the images. Major quad 5 is directly behind quad 0. Its backside is viewed inside the sphere from the positive z-axis.

The respective images have 100 and 121 minor quad faces in each major quad. The even number of minor quads in quad 0 in the first image yields an inner ring of four minor quads centered on the positive z-axis, and the odd number of minor quads in quad 0 in the second image yields a single minor quad centered on the positive z-axis.

A new set of vertices for the convex hull will be generated in the same order as those on the unit sphere in the following manner. Start at the upper-most vertex on the unit sphere, which is on the positive z-axis when n is even or in the first ring of four vertices when n is odd. Let $\underline{n}_s$ represent the normal vector on the sphere at the current vertex. Then search through the entire set of vertices on the convex hull for each vertex whose normal vector $\underline{n}_h$ is within ε angle of $\underline{n}_s$. Average this set of vertices to obtain the representative vertex on the convex hull. This averaging process will be described in more detail later.

From this first vertex on the unit sphere, step to the first or next adjacent vertex in the first ring, which is also the unit sphere's next normal vector $\underline{n}_s$ that will represent the convex hull's next unit normal vector $\underline{n}_h$. Then use the same process as described above to compute a new vertex on the convex hull whose normal vector $\underline{n}_h$ is within ε of $\underline{n}_s$. Repeat this process for all vertex normals in the first ring of vertices on the unit sphere and convex hull, and continue for the remaining rings of vertices on the unit sphere all the way to the bottom ring or vertex on the negative z-axis to finish generating vertices on the entire convex hull surface.

When this step is completed, the unit sphere's topology will have been mapped exactly onto the convex hull, giving it the same number of vertices in the same ring arrangements as the unit sphere. In addition, its set of normal vectors will be identical to the set on the unit sphere. In essence, each arbitrary convex hull's

surface has been divided into six major quads, and each major quad has been divided into n^2 minor quads. And the resulting $6n^2+2$ vertices have been arranged into $3(n-1)$ counterclockwise rings around the positive z-axis. The primary difference is that the shapes of major and minor quads on the arbitrary convex hull are different than those on the unit sphere. But differing shapes of major and minor quads will not affect the algorithms used to compute points and derivatives on the surfaces. Thus one set of algorithms will suffice for all convex hulls generated from the same unit sphere. In essence, the varying curved surfaces on each convex hull will be represented with the same angular uniformity as the non-varying curved surfaces on the unit sphere. In addition, the GJK algorithm used to locate vertices on the unit sphere, based on its unit normal vectors, will also locate vertices on arbitrary convex hulls, based on the same set of unit normal vectors.

As noted above, vertices on the unit sphere are arranged in counterclockwise rings around the z-axis, ranging from the most positive z-values at the top to the most negative values at the bottom. To support the GJK search algorithm, a method is required to locate the vertex on the convex hull's surface whose normal vector is closest to any unit direction vector whose coordinates are given as $[\,x\ y\ z\,]^T$. A hash table will be set up to rapidly accomplish this lookup when performing surface contact detection. Let

$$\begin{aligned} x &= \pm\cos\phi_y\cos\phi_z \Big/ \sqrt{1-\sin^2\phi_y\sin^2\phi_z} \\ y &= \pm\cos\phi_y\sin\phi_z \Big/ \sqrt{1-\sin^2\phi_y\sin^2\phi_z} \quad \text{(quads 4 and 2)} \\ z &= \mp\sin\phi_y\cos\phi_z \Big/ \sqrt{1-\sin^2\phi_y\sin^2\phi_z} \end{aligned} \tag{1.1}$$

$$\begin{aligned} x &= \mp\sin\phi_z\cos\phi_x \Big/ \sqrt{1-\sin^2\phi_z\sin^2\phi_x} \\ y &= \pm\cos\phi_z\cos\phi_x \Big/ \sqrt{1-\sin^2\phi_z\sin^2\phi_x} \quad \text{(quads 1 and 3)} \\ z &= \pm\cos\phi_z\sin\phi_x \Big/ \sqrt{1-\sin^2\phi_z\sin^2\phi_x} \end{aligned} \tag{1.2}$$

and

$$\begin{aligned} x &= \pm\cos\phi_x\sin\phi_y \Big/ \sqrt{1-\sin^2\phi_x\sin^2\phi_y} \\ y &= \mp\sin\phi_x\cos\phi_y \Big/ \sqrt{1-\sin^2\phi_x\sin^2\phi_y} \quad \text{(quads 0 and 5)} \\ z &= \pm\cos\phi_x\cos\phi_y \Big/ \sqrt{1-\sin^2\phi_x\sin^2\phi_y} \end{aligned} \tag{1.3}$$

be three sets of equations representing a unit normal vector where two each of ϕ_x, ϕ_y, or ϕ_z are to be determined from the respective set of equations. Each set of equations in Eq. (1.1), (1.2), or (1.3) is shown in two forms with the respective

upper set of signs applying to quad 4, 1, or 0, and the respective lower set of signs applying to quad 2, 3, or 5. Equation (1.1), (1.2), or (1.3), with the proper set of signs selected, will apply to any unit normal vector on the sphere or convex hull, with the special property that $-\pi/4 \le \phi_x \le \pi/4$, $-\pi/4 \le \phi_y \le \pi/4$, and $-\pi/4 \le \phi_z \le \pi/4$.

A vertex will lie on quad 4 when $x \ge |y|$ and $x \ge |z|$ and on quad 2 when $-x \ge |y|$ and $-x \ge |z|$. In these two cases Eq. (1.1), with the appropriate set of signs, would be selected. A vertex will lie on quad 1 when $y \ge |x|$ and $y \ge |z|$ and on quad 3 when $-y \ge |x|$ and $-y \ge |z|$. In these two cases Eq. (1.2), with the appropriate set of signs, would be selected. And a vertex will lie on quad 0 when $z \ge |x|$ and $z \ge |y|$ and on quad 5 when $-z \ge |x|$ and $-z \ge |y|$. In these two cases Eq. (1.3), with the appropriate set of signs, would be selected.

To better understand how these equations are applied, consider that the eight common corners of the six quads have normal-vector coordinates composed of $\pm\sqrt{3}/3$, and the twelve centers of common edges have two normal-vector coordinates composed of $\pm\sqrt{2}/2$ with the third coordinate zero. Thus the minimum angle between the edge and mean normal vectors of each quad is $\cos^{-1}(\sqrt{2}/2) = \pi/4$ and the maximum angle between corner and mean normal vectors is $\cos^{-1}(\sqrt{3}/3) = 0.3014\pi$. However, as Eqs. (1.1) to (1.3) suggest, the corner normal vectors on the major quads may be reached by two successive orthogonal rotations of $\pm\pi/4$.

Let us first consider an example where two coordinates of a normal vector, say x and y, are each $\sqrt{2}/2$ and $z = 0$. Since $x = y$ are both the largest coordinates, there is a choice between using Eq. (1.1) or (1.2). Choosing Eq. (1.1), the vertex falls into quad 4 with $\phi_y = 0$ and ϕ_z to be determined. Since x and y are positive the first set of signs in Eq. (1.1) are used to obtain

$$\begin{aligned} x &= \sqrt{2}/2 = \cos\phi_z \\ y &= \sqrt{2}/2 = \sin\phi_z \quad (\text{quad } 4) \\ z &= 0 \end{aligned} \tag{1.4}$$

which gives

$$\phi_z = \text{atan2}(y,x) = \text{atan2}(\sqrt{2}/2, \sqrt{2}/2) = \pi/4 \tag{1.5}$$

Similar results would have been obtained using Eq. (1.2). This example illustrates that two different quads will contain the same vertex when it lies on their common edge. Because either quad choice is likely without some heuristics to choose one

over the other, both possibilities will be included in the hash table, and the final choice of vertex will be made using an indirection table. Thus no extra time-consuming tests will be required to pick one quad over the other.

Next consider the direction vector

$$\begin{bmatrix} x & y & z \end{bmatrix}^T = \begin{bmatrix} \sqrt{3}/3 & -\sqrt{3}/3 & \sqrt{3}/3 \end{bmatrix}^T \tag{1.6}$$

In this case each of Eqs. (1.1), (1.2), and (1.3) apply with an appropriate set of signs. Use Eq. (1.2) with the second set of signs to obtain

$$\begin{bmatrix} \sqrt{3}/3 \\ -\sqrt{3}/3 \\ \sqrt{3}/3 \end{bmatrix} = \begin{bmatrix} \sin\phi_z \cos\phi_x \Big/ \sqrt{1-\sin^2\phi_z \sin^2\phi_x} \\ -\cos\phi_z \cos\phi_x \Big/ \sqrt{1-\sin^2\phi_z \sin^2\phi_x} \\ \cos\phi_z \sin\phi_x \Big/ \sqrt{1-\sin^2\phi_z \sin^2\phi_x} \end{bmatrix} \text{ (quad 3)} \tag{1.7}$$

and

$$\begin{aligned} \phi_x &= \text{atan2}(z,-y) = \text{atan2}(\sqrt{3}/3, \sqrt{3}/3) = \pi/4 \\ \phi_z &= \text{atan2}(x,-y) = \text{atan2}(\sqrt{3}/3, \sqrt{3}/3) = \pi/4 \end{aligned} \tag{1.8}$$

The results in Eq. (1.8) are verified by substituting back into Eq. (1.7). In this case three quads share the same vertex at a common corner, so the indirection table will send three different hash-table output results to the same vertex, depending on which of Eq. (1.1), (1.2), or (1.3) was selected.

The next issue is whether there are any cases where Eqs. (1.1) to (1.3) can result in a ϕ_x, ϕ_y, or ϕ_z solution that has one or more values out of their limits, causing one quad to overlap another. Analyzing the method of using two inverse tangent operations, where the largest absolute coordinate in the Eqs. (1.1) to (1.3) is always in the denominator, shows that it is impossible to obtain an angle out of the specified ranges. Thus an efficient and reliable hash-table keyword-coding scheme can be set as described next.

The six major quads 0 through 5 cover the entire surface of the unit sphere or equivalently the convex hull. Let n be a positive integer greater than 0, where each major quad is subdivided into n^2 minor quads and faces with $(n+1)^2$ vertices and associated normal vectors. Thus the total number of minor quads will be $6n^2$ and the total number of unique vertices and normal vectors $6n^2+2$, which is less than $6(n+1)^2$. Every possible vertex and corresponding unit normal vector on the

respective unit sphere or convex hull lies in one or more of the six major quads and is represented by one or more sets of equations in Eqs. (1.1) to (1.3). Thus the first part of each hash-table keyword code will be the character '0' to '5', corresponding to the respective major quad that the vertex and its normal vector lies in. Hash-table keywords are associated with vertex normal vectors because they are the same for the sphere and related convex hulls, while the vertex vectors are generally different.

The second and third components of each hash-table keyword come from the two angles obtained from Eqs. (1.1), (1.2), or (1.3). When Eq. (1.1) for x is used, the angles ϕ_y and ϕ_z are represented in that order, when Eq. (1.2) for y is used, the angles ϕ_z and ϕ_x are represented in that order, and when Eq. (1.3) for z is used, the angles ϕ_x and ϕ_y are represented in that order.

On the unit sphere each major quad is divided equally in its two orthogonal directions by two sets of $n+1$ great circles with poles centered on two orthogonal axes. For quads 4 and 2 the poles are on the $\pm y$ and $\pm z$-axes, for quads 1 and 3 they are on the $\pm z$ and $\pm x$-axes, and for quads 0 and 5 they are on the $\pm x$ and $\pm y$-axes. Note that the above angle subscripts correlate with these poles because they represent the angular rotation of the great circles around the respective axes.

The angle between each adjacent pair of great circles with the same poles is $\pi/(2n)$. Therefore the angle increments in ϕ_x, ϕ_y, and ϕ_z for all vertices in the major quads are exactly $\pi/(2n)$. Thus if ϕ_x, ϕ_y, and ϕ_z are each scaled by $2n/\pi$, their scaled values will be integers ranging from $-n/2$ to $n/2$. To see this, recall that $-\pi/4 \le \phi_x \le \pi/4$, $-\pi/4 \le \phi_y \le \pi/4$, and $-\pi/4 \le \phi_z \le \pi/4$.

However, there is a problem here when n is odd because integer division truncates, and this will bias the quantization process. This problem can be resolved by adding $\pi/4$ to each of the inequalities to obtain $0 \le \phi_x^* \le \pi/2$, $0 \le \phi_y^* \le \pi/2$, and $0 \le \phi_z^* \le \pi/2$. Now scaling these inequalities by $2n/\pi$ gives $0 \le 2n\phi_x^*/\pi \le n$, $0 \le 2n\phi_y^*/\pi \le n$, and $0 \le 2n\phi_z^*/\pi \le n$, which works equally well for odd and even n.

Using this approach, the integer representations for discrete values of the angles will range from 0 to n and require no negative signs. Limiting the magnitude of n to 99, which should be more than enough resolution for any application, each hash-table keyword would be exactly five characters long—one character for the major quad number, 0 through 5, two characters for the first quantized angle, 00 through 99, and two characters for the second quantized angle, 00 through 99. When an angle quantizes to a single integer, a leading 0 is appended to expand it to two characters.

The quadded unit sphere and convex hulls obtained from the unit sphere are each divided into six major quads and $6n^2$ minor quads with a total of $6n^2+2$ unique vertices. But to simplify evaluating hash-table keywords and accessing the nearest vertices on the convex hulls, it is better to support all $(n+1)^2$ vertices on each major quad, so a total of $6(n+1)^2$ keywords will be programmed into the hash-table. Then a $6(n+1)^2$ sized indirection table will be used to point to the $6n^2+2$ unique vertices associated with the same number of unique unit normal vectors.

In order to generate a set of hash-table keywords representing every possible normal vector on the unit sphere and on any convex hull based on that unit sphere, it is necessary to establish precise boundaries for each minor quad on the unit sphere. These boundaries define exactly which quad(s) each point on the unit sphere fall into. Since each point and normal vector on the unit sphere are one and the same, each point on the unit sphere also represents the corresponding unit normal vector on the convex hull, which in turn identifies the corresponding point on the convex hull. A hash-table keyword generated from minor quad boundary angles obtained from a unit direction vector will locate the vertex and associated minor quad on the convex hull whose normal vector is closest to that direction vector. One of the four (three when the vertex is extraordinary) minor quads common to this vertex will contain the point whose normal vector aligns with the direction vector.

The following summarizes how a convex hull vertex would be located by correlating any unit direction vector with one of the hull's unit normal vectors. The unit direction vector has x, y, and z coordinates. The absolutely largest x, y, or z coordinate is used to determine which of Eq. (1.1), (1.2), or (1.3) should be used. Then the sign of that coordinate is used to determine whether the first or second set of signs in the set of equations should be used. From the selected set of equations the two angles are computed using the atan2() function, and $\pi/4$ is added to each result. Next each modified angle is scaled by $2n/\pi$ and rounded to an integer. The two tests on coordinate magnitude and identifying the sign of the largest absolute coordinate have singled out the major quad number between 0 and 5. Finally the quad number and the two scaled and quantized angles are assembled into a five-character keyword, which is entered into the hash table to identify the vertex index. Once the vertex index is known, the four (or three in the eight extraordinary vertex cases) minor quads potentially containing the point are identified.

An example is used to illustrate the process. Assume that $n=20$ so $2n/\pi=12.732$. Then let $\underline{d}=[-0.431, 0.542, -0.721]^T$ represent the unit search direction vector. Since the third component is the absolutely largest coordinate and negative, the vertex is in quad 5 and the second form of Eq. (1.3) is used to obtain

$$\begin{aligned}\phi_x^* &= \text{atan2}(y,-z)+\pi/4 = \text{atan2}(0.542, 0.721)+\pi/4 = 1.430\\ \phi_y^* &= \text{atan2}(-x,-z)+\pi/4 = \text{atan2}(0.431, 0.721)+\pi/4 = 1.324\end{aligned} \tag{1.9}$$

Scaling these two angles with $2n/\pi$ gives 18.21 and 16.86, which round to 18 and 17. Thus the hash-table keyword for looking up the vertex index associated with this direction vector is '51817'. Based on this keyword the hash table returns a unique index into an indirection table, which points to the vertex index associated with the normal vector whose coordinates have decoded to '51817'. Note that this code applies equally to the unit normal vector at the vertex on the unit sphere and at the vertex on the convex hull whose computed unit normal vector coordinate angles scale exactly to 18 and 17. This normal vector on the unit sphere or convex hull is the closest to the direction vector $\underline{d}$. Thus, based on a vertex's associated unit normal vector, the hash table function has directly returned the vertex's index in O(1) time.

The two angles determining the normal vector at the vertex between the minor quads would be

$$\begin{aligned}\phi_x &= 18\pi/(2n)-\pi/4 = 9\pi/20-5\pi/20 = 8\pi/40\\ \phi_y &= 17\pi/(2n)-\pi/4 = 17\pi/40-10\pi/20 = 7\pi/40\end{aligned} \tag{1.10}$$

Substituting these angles into the second form of Eq. (1.3) gives

$$\begin{aligned}x &= -\cos(8\pi/40)\sin(7\pi/40)\Big/\sqrt{1-\sin^2(8\pi/40)\sin^2(7\pi/40)}\\ y &= \sin(8\pi/40)\cos(7\pi/40)\Big/\sqrt{1-\sin^2(8\pi/40)\sin^2(7\pi/40)}\quad \text{quads}\,5\\ z &= -\cos(8\pi/40)\cos(7\pi/40)\Big/\sqrt{1-\sin^2(8\pi/40)\sin^2(7\pi/40)}\end{aligned} \tag{1.11}$$

or

$$\begin{aligned}x &= -0.809*0.5225/0.9517 = -0.4442\\ y &= 0.5879*0.8526/0.9517 = 0.5267\\ z &= 0.809*0.8526/0.9517 = -0.7248\end{aligned} \tag{1.12}$$

Comparing these coordinates of the normal vector at the vertex with the unit direction vector coordinates $\underline{d} = [-0.431, 0.542, -0.721]^T$ shows that the two vectors are quite close to each other. The angle between the two vectors is 0.0315 radians or 1.81°. As will be shown shortly for $n = 20$, half the nominal angle between adjacent unit normal vectors at quad corners is roughly $0.867/n = 0.0434$ radians or 2.48°. At 1.81° the direction vector is definitely closer to the indicated vertex normal vector than to any adjacent one.

Given a unit direction vector, locating a vertex with the closest normal vector on the triangulated surface of a convex hull is somewhat of a challenge. Triangulated surfaces are used here because they are most general and MeshLab always exports triangulated surfaces. In essence MeshLab will import other polygon types, but it will always convert them to triangular representations on export. Let $\underline{n}_s$ represent any one of the vertex normal vectors on the unit sphere, and it is desirable to locate a vertex $\underline{v}_h$ on the convex hull whose normal vector $\underline{n}_h$ most closely aligns with $\underline{n}_s$. The problem is there may be two or more vertices $\underline{v}_h$ with normal vectors $\underline{n}_h$ that closely align with $\underline{n}_s$. Since there can be only one $\underline{v}_h$ associated with each $\underline{n}_s$, the question is, which one should it be?

On convex hulls, when multiple vertices have nearly the same normal vectors, they are either distributed across a nearly flat convex region or along a convex crest or ridge. In either case the average of these vertices can be used to represent the desired vertex because the average must lie in the interior of the convex region containing the vertices. This holds because the surface of a convex hull is everywhere convex, which means it cannot contain even one small non-convex patch.

The incremental angular distance between great circles was set to $\pi/(2n)$ when dividing up the unit sphere. Where the minor quads are nearly square the maximum angular distance between adjacent vertex normal vectors is roughly $(\sqrt{2}/2)\pi/(2n)$, and near the extraordinary vertex points where the minor quads are most distorted, the maximum angular distance is roughly $(3/2)\pi/(2n)$. Since we want to locate convex hull vertices $\underline{v}_h$ whose normal vectors $\underline{n}_h$ are clustered around the unit sphere's normal vector $\underline{n}_s$, a maximum angular distance of $(\sqrt{2}+3)\pi/(16n)$ will be used to determine when $\underline{n}_h$ is sufficiently close to $\underline{n}_s$. This value is obtained by taking half the average of the two angles.

Since generating the set of vertices on the convex hull is done only once, no attempt is made to optimize the process. Thus normal vectors $\underline{n}_h$ for the entire set of convex hull vertices are compared to each normal vector $\underline{n}_s$. For each normal vector $\underline{n}_h$ that falls within the angular distance $(\sqrt{2}+3)\pi/(16n) = 0.867/n$ of $\underline{n}_s$, its corresponding vertex vector $\underline{v}_h$ is added to a register vector and the count increased by one. The convex hull's representative vertex $\underline{v}_h$ for $\underline{n}_s$ is then obtained by dividing the register by the number of matches. This averaging process works particularly well when the convex hull's triangles are much smaller than the unit sphere's quads.

Consider the case when $n = 20$ and the angular increments between great circles are $\pi/(2n) = \pi/40 = 0.0785$ radians. The maximum angular distance between normal vectors $\underline{n}_h$ on the convex hull and $\underline{n}_s$ on the unit sphere for a match would be $0.867/n = 0.867/20 = 0.04335$ radians. This is a little less than 2.5°. Thus any

vertex on the convex hull whose normal vector is within about 2.5° of $\underline{n}_s$ will contribute to the vertex $\underline{v}_h$ whose normal vector $\underline{n}_h$ will be $\underline{n}_s$.

Sorting out the vertices on the convex hull that contribute to $\underline{v}_h$ would be done as follows. For each candidate vertex $\underline{v}_h$, evaluate $\underline{n}_s^T \underline{n}_h = \cos\theta_{sh}$ where θ_{sh} is the angle between the two vectors. For small angles, $\cos\theta_{sh} \simeq 1 - \theta_{sh}^2 / 2$ so $\theta_{sh}^2 / 2 \simeq 1 - \underline{n}_s^T \underline{n}_h$ or $\theta_{sh} \simeq \sqrt{2(1 - \underline{n}_s^T \underline{n}_h)}$. Alternately

$$\begin{aligned} \underline{n}_s^T \underline{n}_h &> 1 - \theta_{sh}^2 / 2 \\ &= 1 - (0.867 / n)^2 / 2 \\ &= 1 - 0.376 / n^2 \\ &= 1 - 0.00094 \\ &= 0.9991 \end{aligned} \tag{1.13}$$

when $n = 20$. Thus it is sufficient to compare vector inner products to a positive constant to decide which vertices on the convex hull will participate in each vertex definition. Averaging vertices over a triangulated surface will introduce a small amount of error, but that error will be reduced as the average size of triangles representing the convex hull is reduced. Thus if more accuracy is needed the resolution can be increased by further subdividing the triangulated surface. It is assumed that each convex hull surface has been sufficiently subdivided to achieve the desired accuracy.

Once the unit sphere's quadded surface topology has been projected onto the convex hull's surface, any point on its surface may be evaluated with C^2 continuity, except around the eight extraordinary points where the valence is 3. The general method of evaluating points and their first and second derivatives is straightforward and efficient when the valence is 4. Away from the extraordinary points and after setting up the local u and v spline variables, it takes 45 multiplications and 45 additions to evaluate a point on the surface when the valence is 4.

To see how all this will be used in practice let us consider modeling contact between two arbitrary convex hulls, labeled 1 and 2. Each convex hull has the surface topology of a quadded sphere, but the values for n_1 and n_2, the number of minor quad edges across each major quad, may not be the same if different quadded unit spheres are used to set their topologies. The primary information used in contact detection and modeling is the common normal vector between the two surfaces. Given this common normal vector, hash-table functions will return the vertex location on each surface whose normal vector most closely matches it. When the valence is 4 the desired point will be in one of the four quads surrounding the

vertex, and when the valence is 3 the point will be in one of the three quads surrounding the vertex.

When the two convex hulls are within shouting distance of each other an iterative GJK algorithm will generate a unit direction vector used to establish the approximately closest vertices on the two surfaces. When the separation distance is sufficiently large so that contact is impossible, it is sufficient to return the closest vertex points on the surfaces, which would be equivalent to modeling contact between polyhedrons. But when the surfaces are near or in contact, the bicubic B-splines must be applied to compute actual points on the curved surfaces. Since the surfaces are represented by bicubic B-splines, and the constraint equations are even higher than bicubic, Newton-Raphson iteration is used to zero in on the exact contact locations. We assume that the GJK algorithm has identified a pair of vertices on the two convex hulls close enough to each other that Newton-Raphson iteration will converge.

To solve for the contact points and penetration, we first describe the constraint equations and Jacobian matrix and then show how the quantities would be evaluated from the bicubic B-splines. Assume that an orthogonal Cartesian coordinate frame numbered 0 is attached to ground and various quantities are measured relative to this frame. Let 0 represent the origin of this frame. Let Cartesian frames 1 and 2 be attached to the two objects with convex hulls 1 and 2 attached to them, and let 1 and 2 represent the origins of these frames. Points on the respective objects and convex hulls are measured relative to the origins of the respective Cartesian frames, and coordinates on these objects are expressed in their coordinate frames.

The loop equations across the two surfaces are

$$\underline{t}^{\,0}_{pq} = \underline{t}^{\,0}_{02} + \underline{R}^{02}\,\underline{t}^{\,2}_{2q} - \underline{t}^{\,0}_{01} - \underline{R}^{01}\,\underline{t}^{\,1}_{1p} \tag{1.14}$$

where the three by one coordinate matrices $\underline{t}^{\,0}_{01}$ and $\underline{t}^{\,0}_{02}$ represent the position of frame origins expressed in frame 0 coordinates. The three by three direction cosine matrices $\underline{R}^{01}$ and $\underline{R}^{02}$ represent the orientation of frames 1 and 2 relative to frame 0. The three by one coordinate matrices $\underline{t}^{\,1}_{1p}(u_1, v_1)$ and $\underline{t}^{\,2}_{2q}(u_2, v_2)$, expressed in the local frames, give the position of the points on the respective surfaces as a function of the local u_1, v_1, u_2, and v_2 bicubic B-spline variables. These are the four quantities that must be solved for applying Newton-Raphson iteration to four constraint equations presented next.

The vector $\underline{t}^{\,0}_{pq}$ represents a vector from a point on convex hull 1 to another point on convex hull 2. When the constraints are satisfied this vector must be orthogonal to both surfaces. Setting dot products between this normal vector and tangent vectors

at points p and q on the respective surfaces to zero implements the four constraint equations. Assume for now that $\underline{t}^1_{1p}$ and $\underline{t}^2_{2q}$ and that their first and second partial derivatives, $\underline{t}^1_{u1}$, $\underline{t}^1_{v1}$, $\underline{t}^2_{u2}$, $\underline{t}^2_{v2}$, $\underline{t}^1_{u1u1}$, $\underline{t}^1_{v1v1}$, $\underline{t}^1_{u1v1} = \underline{t}^1_{v1u1}$ $\underline{t}^2_{u2v2}$, $\underline{t}^2_{v2v2}$, and $\underline{t}^2_{u2v2} = \underline{t}^2_{v2u2}$ have been evaluated from the bicubic B-splines. The vectors $\underline{t}^1_{u1}$, $\underline{t}^1_{v1}$, $\underline{t}^2_{u2}$, and $\underline{t}^2_{v2}$ represent the four tangents to the surfaces, and the remaining six vectors represent partial derivatives of the tangent vectors. These quantities are used to set up the constraint equations and Jacobian matrix is follows. The four constraint equations are

$$\begin{aligned}
\phi_{u1} &= \underline{t}^{1T}_{u1}\underline{R}^{01T}\underline{t}^0_{pq} = \underline{t}^{1T}_{u1}\underline{R}^{01T}(\underline{t}^0_{02} + \underline{R}^{02}\underline{t}^2_{2q} - \underline{t}^0_{01} - \underline{R}^{01}\underline{t}^1_{1p}) = 0 \\
\phi_{v1} &= \underline{t}^{1T}_{v1}\underline{R}^{01T}\underline{t}^0_{pq} = \underline{t}^{1T}_{v1}\underline{R}^{01T}(\underline{t}^0_{02} + \underline{R}^{02}\underline{t}^2_{2q} - \underline{t}^0_{01} - \underline{R}^{01}\underline{t}^1_{1p}) = 0 \\
\phi_{u2} &= \underline{t}^{2T}_{u2}\underline{R}^{02T}\underline{t}^0_{pq} = \underline{t}^{2T}_{u2}\underline{R}^{02T}(\underline{t}^0_{02} + \underline{R}^{02}\underline{t}^2_{2q} - \underline{t}^0_{01} - \underline{R}^{01}\underline{t}^1_{1p}) = 0 \\
\phi_{v2} &= \underline{t}^{2T}_{v2}\underline{R}^{02T}\underline{t}^0_{pq} = \underline{t}^{2T}_{v2}\underline{R}^{02T}(\underline{t}^0_{02} + \underline{R}^{02}\underline{t}^2_{2q} - \underline{t}^0_{01} - \underline{R}^{01}\underline{t}^1_{1p}) = 0
\end{aligned} \tag{1.15}$$

For Newton-Raphson iteration the quantities $\underline{t}^0_{01}$, $\underline{t}^0_{02}$, $\underline{R}^{01}$ and $\underline{R}^{02}$ may be considered constant, with the point and tangent vectors variable, depending on u_1, v_1, u_2, and v_2. Partial derivatives of the four constraint equations with respect to the four variables yield the Jacobian matrix

$$J = \begin{bmatrix}
\underline{t}^{1T}_{u1u1}\underline{R}^{01T}\underline{t}^0_{pq} - \underline{t}^{1T}_{u1}\underline{t}^1_{u1} & \underline{t}^{1T}_{u1v1}\underline{R}^{01T}\underline{t}^0_{pq} - \underline{t}^{1T}_{u1}\underline{t}^1_{v1} & \underline{t}^{1T}_{u1}\underline{R}^{12}\underline{t}^2_{u2} & \underline{t}^{1T}_{u1}\underline{R}^{12}\underline{t}^2_{v2} \\
\underline{t}^{1T}_{v1u1}\underline{R}^{01T}\underline{t}^0_{pq} - \underline{t}^{1T}_{v1}\underline{t}^1_{u1} & \underline{t}^{1T}_{v1v1}\underline{R}^{01T}\underline{t}^0_{pq} - \underline{t}^{1T}_{v1}\underline{t}^1_{v1} & \underline{t}^{1T}_{v1}\underline{R}^{12}\underline{t}^2_{u2} & \underline{t}^{1T}_{v1}\underline{R}^{12}\underline{t}^2_{v2} \\
-\underline{t}^{2T}_{u2}\underline{R}^{12T}\underline{t}^1_{u1} & -\underline{t}^{2T}_{u2}\underline{R}^{12T}\underline{t}^1_{v1} & \underline{t}^{2T}_{u2u2}\underline{R}^{02T}\underline{t}^0_{pq} + \underline{t}^{2T}_{u2}\underline{t}^2_{u2} & \underline{t}^{2T}_{u2v2}\underline{R}^{02T}\underline{t}^0_{pq} + \underline{t}^{2T}_{u2}\underline{t}^2_{v2} \\
-\underline{t}^{2T}_{v2}\underline{R}^{12T}\underline{t}^1_{u1} & -\underline{t}^{2T}_{v2}\underline{R}^{12T}\underline{t}^1_{v1} & \underline{t}^{2T}_{v2u2}\underline{R}^{02T}\underline{t}^0_{pq} + \underline{t}^{2T}_{v2}\underline{t}^2_{u2} & \underline{t}^{2T}_{v2v2}\underline{R}^{02T}\underline{t}^0_{pq} + \underline{t}^{2T}_{v2}\underline{t}^2_{v2}
\end{bmatrix} \tag{1.16}$$

where $\underline{R}^{12} = \underline{R}^{01T}\underline{R}^{02}$ and the remaining quantities are obtained from the bicubic B-splines. Equations (1.15) and (1.16) contain all the information needed to iteratively update the two points on the surfaces using Newton-Raphson iteration.

Next it is illustrated how the point and its partial derivatives on the first convex hull are evaluated using the parameters u_1 and v_1. The same procedures would be applied on the second convex hull so it won't be illustrated. At the moment we assume that the point is not near an extraordinary point so the regular method of evaluating a bicubic B-spline and its partial derivatives will be applied. The special cases will be discussed later. For discussion purposes assume that indices i and j identify a control point at the vertex on the corner of the quad containing the point of interest. The bicubic B-spline requires 16 control points surrounding this quad. Let

$$C_{ij}^{1}=\begin{bmatrix} \underline{c}_{i+2,j-1}^{1} & \underline{c}_{i+2,j}^{1} & \underline{c}_{i+2,j+1}^{1} & \underline{c}_{i+2,j+2}^{1} \\ \underline{c}_{i+1,j-1}^{1} & \underline{c}_{i+1,j}^{1} & \underline{c}_{i+1,j+1}^{1} & \underline{c}_{i+1,j+2}^{1} \\ \underline{c}_{i,j-1}^{1} & \underline{c}_{ij}^{1} & \underline{c}_{i,j+1}^{1} & \underline{c}_{i,j+2}^{1} \\ \underline{c}_{i-1,j-1}^{1} & \underline{c}_{i-1,j}^{1} & \underline{c}_{i-1,j+1}^{1} & \underline{c}_{i-1,j+2}^{1} \end{bmatrix} \tag{1.17}$$

represent coordinates of the sixteen control points. Then let

$$M=\frac{1}{6}\begin{bmatrix} 1 & 4 & 1 & 0 \\ -3 & 0 & 3 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix} \tag{1.18}$$

represent the B-spline shape matrix and $\underline{u}_1=[1\ u_1\ u_1^2\ u_1^3]^T$ the Bernstein basis. Then

$$\underline{u}_1^T M C_{ij}^1 \tag{1.19}$$

evaluates four control points that may be used with M^T and $\underline{v}_1=[1\ v_1\ v_1^2\ v_1^3]^T$ to obtain the desired approximated point

$$\underline{t}_{1p}^{1}=\underline{u}_1^T M C_{ij}^1 M^T \underline{v}_1 \tag{1.20}$$

There is a problem with Eqs. (1.19) and (1.20) because the column dimension of M doesn't conform with the row dimension of C_{ij}^1, and Eq. (1.20) would imply that $\underline{t}_{1p}^{1}$ is a scalar. To fix this we split C_{ij}^1 into three 4 by 4 matrices with the x, y, and z coordinates separated. Designate these three matrices as C_{xij}^1, C_{yij}^1, and C_{zij}^1. With this we correct Eq. (1.20) to

$$\underline{t}_{1p}^{1}=\begin{bmatrix} \underline{u}_1^T M C_{xij}^1 M^T \underline{v}_1 \\ \underline{u}_1^T M C_{yij}^1 M^T \underline{v}_1 \\ \underline{u}_1^T M C_{zij}^1 M^T \underline{v}_1 \end{bmatrix} \tag{1.21}$$

It is expensive to carry out all these multiplications every time, so the constant matrices can be evaluated once and stored as

$$X_{ij}^1 = MC_{xij}^1 M^T$$
$$Y_{ij}^1 = MC_{yij}^1 M^T \qquad (1.22)$$
$$Z_{ij}^1 = MC_{zij}^1 M^T$$

so that Eq. (1.21) reduces to

$$\underline{t}_{1p}^1 = \begin{bmatrix} \underline{u}_1^T X_{ij}^1 \underline{v}_1 \\ \underline{u}_1^T Y_{ij}^1 \underline{v}_1 \\ \underline{u}_1^T Z_{ij}^1 \underline{v}_1 \end{bmatrix} \qquad (1.23)$$

The first and second partial derivatives of $\underline{t}_{1p}^1$ with respect to u_1 and v_1 are easily obtained from Eq. (1.23) by first setting the partial derivative matrices $\underline{u}_{1u} = [0\ 1\ 2u_1\ 3u_1^2]^T$, $\underline{v}_{1v} = [0\ 1\ 2v_1\ 3v_1^2]^T$, $\underline{u}_{1uu} = [0\ 0\ 2\ 6u_1]^T$, and $\underline{v}_{1vv} = [0\ 0\ 2\ 6v_1]^T$. Algorithms based on Horner's method can be set up to efficiently evaluate the vectors and their partial derivatives, taking advantage of constants, sparsity, and repetition. They are presented later.

A significant bookkeeping issue is determining which minor quad a point falls into and transitioning the B-spline parameters from one minor quad to the next. It was noted that one vertex is common to four adjacent minor quads, (three adjacent minor quads when the vertex has valence 3) and when a point is associated with that vertex, it must be determined into which of the four (three) minor quads it falls. Each minor quad face can have associated with it some transition rules to help identify the correct minor quad for the point. These rules are presented later.

The significant issue is when the GJK algorithm identifies the initial vertex on each of the two convex hulls, and no values for the local u_1, v_1, u_2, and v_2 B-spline variables have been established. The only information available at the time is the unit direction vector from the GJK algorithm to associate with a particular normal vector on each of the two convex hulls. The lean of this direction vector away from each normal vector can be used to determine in which of the four (three) quads each point lies. The direction vector's lean into the respective quads can be used to obtain a first approximation of u_1, v_1, u_2, and v_2 to improve Newton-Raphson iteration convergence.

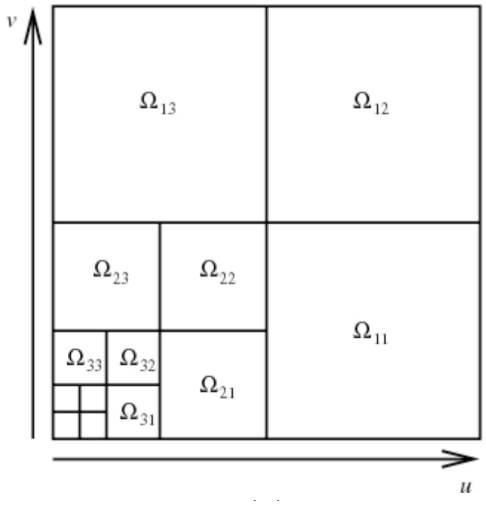
Now we consider the situation where a point moves from one quad into another. The limits on each B-spline variable in a quad are $0 \le u, v < 1$. When a variable goes outside either of its limits, the point will have moved across a boundary into an adjacent quad. A nice feature of C^2 continuity is that displacements, and first and second partial derivatives on the surface are continuous across boundaries between adjacent quads. Thus each quad is assigned pointers to its four adjacent quads so

that the correct quad can be directly accessed when either variable goes outside a limit.

In situations where both variables simultaneously go outside a limit, the transition would take place in two steps. First u would transition into a new quad, where v is still out of limits. Then from the new quad, v would transition into a second new quad, so that u and v are both back within limits. Since all quantities in the constraints and Jacobian matrix are continuous across the quad boundaries, the Newton-Raphson iteration algorithm will not be disrupted by these transitions, which are just bookkeeping issues.

Evaluating points using bicubic B-splines is fine except on any extraordinary quad face containing an extraordinary valence-3 vertex, where only seven surrounding quad faces are available instead of the usual eight. The regular bicubic B-spline algorithm requires 16 control points to function properly, and there will be an insufficient number (14) in this case to set up the necessary equations. For this reason quad subdivision algorithms like Catmull-Clark were invented to isolate the extraordinary vertices into small enough regions so that they could be ignored. The subdivision process produces smaller and smaller quads until the smallest subdivision adequately represents the surface. This method works fine when a large number of GPUs are available to carry out the subdivision operations in parallel, but contact modeling requires subdivision only at a single point, not over the entire face.

When n is moderate in the 20 to 40 range it may be more efficient to apply several Catmull-Clark subdivisions to obtain the desired ε point accuracy on the surface. Since contact points generally move smoothly over convex hull surfaces, the recently evaluated subdivision data for a given extraordinary face can be stored temporarily for reuse until the point has moved out of the area. Thus amortizing the subdivision cost over numerous close-by point and derivative operations can make the Catmull-Clark subdivision algorithm for irregular faces competitive with the analytic bicubic B-spline method for regular faces.



If a unit sphere is quadded with $n = 20$, it will have $6n^2 = 2400$ faces with 24 or 1% of them extraordinary. Thus roughly 99% of the surface area of the unit sphere or of any convex hull whose topology is represented by that unit sphere will be ordinary and represented by efficient ordinary bicubic B-spline approximation using Horner's method. Each of the remaining 24 extraordinary quads has the origin of its local u-v coordinate system at one extraordinary vertex as illustrated in the above image. One Catmull-Clark subdivision step will split the extraordinary quad into four quads of roughly 1/4th the area each. The three quads labeled Ω_{11}, Ω_{12}, and Ω_{13} are all ordinary, and the area they cover can be approximated using Horner's method.

The remaining quarter area of the quad is again extraordinary and subdivided into Ω_{21}, Ω_{22}, and Ω_{23} ordinary quads plus a fourth extraordinary quad. This now leaves 1/16th of the area extraordinary. Five levels of subdivision would leave 1/1024th of the area extraordinary. Applying five levels of subdivision to the 24 extraordinary quads would leave about 0.001% of the surface area extraordinary. Assume that the mean radius of a convex hull is one meter so its surface area is roughly $4\pi m^2$. Then the mean area of the remaining extraordinary quads would be $4\pi / (2400 * 1024) = 5\,\mathrm{mm}^2$ and the mean edge length $2\,\mathrm{mm}$.

Thus a two-meter diameter convex object represented by 2400 quads (24 of them extraordinary), when Catmull-Clark subdivided five times, would have a maximum wavelength of $2\,\mathrm{mm}$ roughly 0.001% of the time. As the above figure suggests, subdividing 24 extraordinary quads five times will replace them with 15*24=360 ordinary quads and 24 extraordinary quads, giving a total of 2736 ordinary quads (when $n = 20$) now covering 99.999% of the surface area. The remaining roughly $1\,\mathrm{cm}^2$ surface area ($5\,\mathrm{mm}^2$ per extraordinary quad) can be represented by the nearest adjacent ordinary quads.

Inspection of the quadded sphere reveals that three extraordinary quads share each extraordinary vertex (eight of them), so after five subdivisions there would remain eight clusters of three adjacent extraordinary quads on the surface, each covering about 0.0001% of the surface. Thus at a small expense of increasing the number of ordinary quads by about 14% and adding logic to shift and scale the u and v variables for the subdivided ordinary quads, the huge overhead of Stam's algorithm can be avoided.

There seems to be no reference of anyone using the regular bicubic B-spline algorithm to evaluate points on the regular Ω_{11}, Ω_{12}, Ω_{13} Ω_{21}, Ω_{22}, and Ω_{23}, etc. quads, but instead it appears they continue applying Catmull-Clark subdivision to them as well to obtain a uniform subdivision at the finest level over the entire original quad surface. This would be fine when rendering the entire surface using GPUs, but for contact modeling only a single point needs to be evaluated somewhere

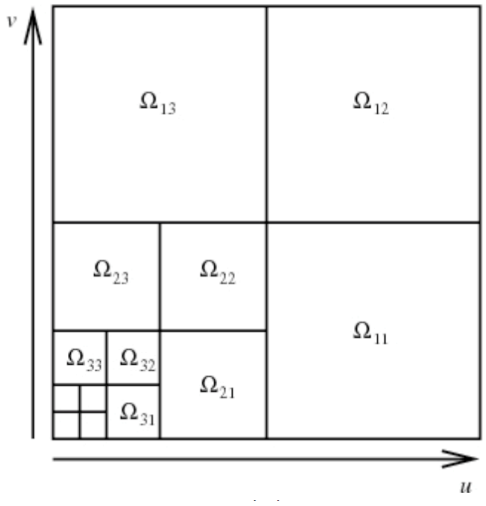
on the surface, but not at every point. Thus the small number of subdivisions can be precomputed once and stored. Then for the above example of $n = 20$ it is a simple matter of determining which of the 2736 ordinary quads to use. For any unit direction vector set by the GJK algorithm, the hash table will go directly to the vertex whose normal vector most closely aligns with it. And once there Newton-Raphson iteration will be used to walk points across mating surfaces of convex hulls. The primary issues related to this task are transitioning points across boundaries between the 2736 ordinary quads on each surface (assuming $n = 20$) and redefining the u and v variables, which are the four independent quantities (for mating surfaces) used by the Newton-Raphson algorithm. To accomplish this, each regular quad has a data structure containing rules for transitioning to its neighbors. Where a quad has no subdivided neighbors or is not a subdivided quad, the number of transition rules is exactly four and they are easy to implement. Otherwise the number of transition rules may be more than four and they may include shifting and scaling the u and v variables as suggested by the above subdivision image. Getting this data structure and supporting algorithms set up is what I'm working on now.

All ordinary quads are set up for the origin of the u-v coordinate system to be located at the first of four vertices. In quads 0 through 4 all vertices are arranged counterclockwise so the vertex and face normals point outward, and in quad 5, which is viewed from inside, the vertices are arranged clockwise. Eight quads surround each ordinary quad, where zero to two of them may be extraordinary. The four surrounding quads, each with just one vertex in common, contain the sixteen support vertices for the regular central quad. The other four surrounding quads, each with one edge in common, are the primary regions where a point will transition to or from when leaving or entering the central quad. Rules for changing variables when transitioning are presented later. Nearly all transitions between minor quads within each major quad are between u-v coordinate systems with the same orientation. The only exceptions are transitions between some ordinary and extraordinary quads. All other transitions between minor quads, where the angles between u-v coordinate systems may be 90°, 180°, and 270°, are across edges of adjacent major quads. The number of these transitions is relatively small, and their special sets of rules are summarized later.

The eight extraordinary vertices are special in that their valences are three, and the 24 extraordinary minor quads are special in that they each contain one extraordinary vertex. In addition, the origin of each extraordinary quad's u-v coordinate system is placed at its extraordinary vertex. When the extraordinary vertex is first in the face list, the origin of the u-v coordinate system will be the same as for all ordinary quads. Otherwise it will be rotated 90°, 180°, or 270° to the second, third, or fourth vertex in the list. These special u-v coordinate system rotations simply require using different sets of u and v variable transitions between some adjacent quads. The quad face lists contain sufficient information to set up all projected control point sets and to define all transition rules.

Two forms of transition information, sending and receiving, take place across edges between adjacent minor quad faces. The sending face first determines which u or v variable is out of bounds and in which direction, and from that information it determines which adjoining face must receive the transitioned variables. Then the receiving face takes over, converting the u and v quantities from the sending face to its own internal representation. These transition conversions may consist of two major parts. The first part accounts for the 0°, 90°, 180°, or 270° rotation of the receiving u-v coordinate system relative to the sending u-v coordinate system, and it accounts for the common edge across which the u and v variable data is sent. The second form of transition accounts for whether or not the sending and/or receiving face has been subdivided. The u and v variables always range between 0 and 1 within the boundaries of each minor quad, but when a quad has been subdivided, the u and v variables within its subdivided quads require different shifting and scaling rules to evaluate points them. However, in these cases the rotation angle between u-v coordinate systems attached to the subdivided quads is always 0°.

General transitions can be from larger to smaller or smaller to larger faces, so the best way to handle them is for the parent face to transition the internal u and v data between its own subdivided faces or between it and other faces. Each subdivided parent quad knows the size and location of all subdivided quads within it and thus knows how to transition its outgoing and incoming u and v data to and from its descendant quads. Having each parent quad contain its own u and v transition rules will substantially simplify interfacing all quads.



The above figure shows a parent quad face subdivided four times, yielding successively smaller quads. If a parent quad were subdivided m times it would consist of $3m$ ordinary quads and one extraordinary quad containing one extraordinary vertex at the origin of its u-v coordinate system. Each subdivided quad will have its own local u-v coordinate system oriented with the parent quad's coordinate system. With the exception of the remaining extraordinary quad, all subdivided quad origins have been shifted from the parent quad's origin, which is

located at the extraordinary vertex. In addition, the local u and v variables within each subdivided quad have been shifted and scaled so that their local arguments take on values between 0 and 1. This local shifting and scaling process will make it easier to keep track of when a u or v variable goes outside its limits and must be transitioned to an adjacent subdivided or parent quad.

Three extraordinary faces surround each of the eight valence-3 extraordinary vertices. Each of the 24 extraordinary faces will be subdivided m times at an additional storage cost of $288m$ extra vertices. Thus four subdivisions would require an additional 1152 vertices. For $6n^2 = 2400$ vertices when $n = 20$ this represents about a 50% increase in data storage to achieve rapid evaluation of all and partial derivatives on all but about 0.0001% of the surface in the immediate area around the eight extraordinary vertices. On a unit sphere this equates to a wavelength of about 0.002 units. If the radius of the sphere were one meter this would correspond to a wavelength of about two millimeters.

For `EvalSurf()` to work properly the values of u and v should not be smaller than some positive ε related to the desired surface resolution. This was discussed earlier and essentially means that a point roughly ε away from the extraordinary vertex is taken to represent any point closer to the vertex. Vertices on the convex hull surfaces were arranged so the angles between adjacent normal vectors would be uniform. Thus partial derivatives near the extraordinary vertices are relatively constant, so the Newton-Raphson iteration algorithm should still work there. The iteration convergence tolerance may be set to depend on ε.

```
void EvalSurf ( double P[3], double u, double v, point *Cp ) {
  /* determine in which domain the parameter lies */
  int n = floor(min(-log2(u),-log2(v)))+1;
  double pow2 = pow(2,n-1); /* Store pow2 as array of pow2n[i] = 2^i */
  u *= pow2;
  v *= pow2;
  if ( v < 0.5 ) {
    k=0;
    u=2*u-1; /* u += u-1; */
    v=2*v;   /* v += v; */
  }
  else if ( u < 0.5 ) {
    k=2;
    u=2*u;   /* u += u; */
    v=2*v-1; /* v += v-1; */
  }
  else {
    k=1;
    u=2*u-1; /* u += u-1; */
    v=2*v-1; /* v += v-1; */
```

```
  }
  /* Now evaluate the surface */
  P[0] = P[1] = P[2] = 0.0;
  for ( i=0; i<14; i++ ) {
    /* Store pow(eigen.L[i],n-1) in a two-dimensional array.
       Set int array of number of terms to use for each n.
       EvalSpline() doesn't know about n or k, so final scaling must
       be done in this function.
       First partial derivatives use pow2n[n] for 2^n.
       Second partial derivatives use pow4n[n] for 2^(2n).
    */
    double sf = pow(eigen.L[i],n-1) * EvalSpline(eigen.x[i][k], u, v);
    P[0] += sf*Cp[i][0];
    P[1] += sf*Cp[i][1];
    P[2] += sf*Cp[i][2];
  }
}


double EvalSpline ( double *C, double u, double v ) {
  double s0 = C[ 0] + u*(C[ 1] + u*(C[ 2] + u*C[ 3]));
  double s1 = C[ 4] + u*(C[ 5] + u*(C[ 6] + u*C[ 7]));
  double s2 = C[ 8] + u*(C[ 9] + u*(C[10] + u*C[11]));
  double s3 = C[12] + u*(C[13] + u*(C[14] + u*C[15]));
  return ( s0 + v*(s1 + v*(s2 * v*s3)) );
}
```

Since first and second partial derivatives are required to evaluate the constraint equations and Jacobian matrix, it will be beneficial to combine their evaluation with the surface using a modified version of Horner's rule. Consider a cubic polynomial

$$\begin{aligned} y &= a_0 + a_1 x + a_2 x^2 + a_3 x^3 = a_0 + x(a_1 + x(a_2 + x(a_3))) \\ y' &= a_1 + a_2 x + a_3 x^2 = a_1 + x(2a_2 + x(3a_3)) \\ y'' &= 2a_2 + x(6a_3) \end{aligned} \tag{1.24}$$

Horner's rule provides a means to recursively evaluate these quantities as

$$
\begin{aligned}
&y = y' = y'' = a_3 \\
&y = a_2 + yx = a_2 + a_3 x \\
&y' = y + y'x = (a_2 + a_3 x) + a_3 x = a_2 + 2a_3 x \\
&y'' = y' + y''x = (a_2 + 2a_3 x) + a_3 x = a_2 + 3a_3 x \\
&y = a_1 + yx = a_1 + (a_2 + a_3 x)x = a_1 + a_2 x + a_3 x^2 \\
&y' = y + y'x = (a_1 + a_2 x + a_3 x^2) + (a_2 + 2a_3 x)x = a_1 + 2a_2 x + 3a_3 x^2 \\
&y = a_0 + yx = a_0 + (a_1 + a_2 x + a_3 x^2)x = a_0 + a_1 x + a_2 x^2 + a_3 x^3 \\
&y'' = y'' + y'' = (a_2 + 3a_3 x) + (a_2 + 3a_3 x) = 2a_2 + 6a_3 x
\end{aligned}
\tag{1.25}
$$

Equation (1.25) shows that six multiplications and seven additions are required to evaluate the cubic equation and its first and second partial derivatives.

A similar process can be applied to bicubic B-splines representing a surface with C^2 continuity. Let u and v represent two parameters of a quartic patch where

$$
c = \begin{bmatrix} 1 & u & u^2 & u^2 \end{bmatrix} \begin{bmatrix} c_0 & c_4 & c_8 & c_{12} \\ c_1 & c_5 & c_9 & c_{13} \\ c_2 & c_6 & c_{10} & c_{14} \\ c_3 & c_7 & c_{11} & c_{15} \end{bmatrix} \begin{bmatrix} 1 \\ v \\ v^2 \\ v^3 \end{bmatrix} = \begin{bmatrix} s_0 & s_1 & s_2 & s_3 \end{bmatrix} \begin{bmatrix} 1 \\ v \\ v^2 \\ v^3 \end{bmatrix} \tag{1.26}
$$

Equation (1.26) shows that five cubic equations are required to evaluate c. Note that the cubic equations s_0 to s_3 depend only on u and only the final equation

$$
c = s_0 + v(s_1 + v(s_2 + v(s_3))) \tag{1.27}
$$

depends on u and v. At the same time evaluating s_0 to s_3 and c, we also want to evaluate the partial derivatives c_u, c_v, c_{uu}, c_{vv}, and $c_{uv} = c_{vu}$ using Horner's rule. First apply Horner's rule to s_0 to s_3 as they are being evaluated to simultaneously obtain s_{0u} to s_{3u} and s_{0uu} to s_{3uu}. Then by taking partial derivatives of Eq. (1.27) with respect to u it follows that

$$
\begin{aligned}
c_u &= s_{0u} + v(s_{1u} + v(s_{2u} + v(s_{3u}))) \\
c_{uu} &= s_{0uu} + v(s_{1uu} + v(s_{2uu} + v(s_{3uu})))
\end{aligned}
\tag{1.28}
$$

Horner's rule can also be applied while evaluating Eq. (1.27) to simultaneously obtain

$$
\begin{aligned}
c &= s_0 + v(s_1 + v(s_2 + v(s_3))) \\
c_v &= s_1 + v(2s_2 + v(3s_3)) = s_1 + 2s_2 v + 3s_3 v^2 \\
c_{vv} &= 2s_2 + 6s_3 v
\end{aligned}
\tag{1.29}
$$

All that remains is to evaluate c_{uv}, which may be generated simultaneously with c_u using Horner's rule as

$$
\begin{aligned}
c_u &= s_{0u} + v(s_{1u} + v(s_{2u} + v(s_{3u}))) = s_{0u} + s_{1u} v + s_{2u} v^2 + s_{3u} v^3 \\
c_{uv} &= s_{1u} + v(2s_{2u} + v(3s_{3u})) = s_{1u} + 2s_{2u} v + 3s_{3u} v^2
\end{aligned}
\tag{1.30}
$$

The second equation in Eq. (1.28), the three equations in Eq. (1.29), and the two equations in Eq. (1.30) represent the additional operations required to evaluate the bicubic B-spline and its partial derivatives. The total number of multiplications and additions, including those for s_0 to s_3, s_{0u} to s_{3u}, and s_{0uu} to s_{3uu}, is 38 and 43.

This would be the number of operations associated with the x, y, or z coordinate of a point on the surface. Thus 114 multiplications and 129 additions would be required for one point and its partial derivatives. This compares with 18 multiplications and 21 additions for a single point on a curve.

The above `EvalSpline()` function must be modified to return not only one coordinate of the point but also the first and second partial derivatives of with respect to the u and v variables.

```
void EvalSpline ( double *C, double u, double v, double r[6] ) {
  /* r[6] contains y, y_u, y_v, y_uu, y_uv, y_vv */
  double s[4][3];
  /* s[i][0], s[i][1], s[i][2], contain s_i, s_iu, s_iuu */
  for ( int i=0; i<3; i++) {
    s[i][0] = s[i][1] = s[i][2] = C[4*i+3];
    s[i][0] = C[4*i+2] + s[i][0]*u;
    s[i][1] = s[i][0]  + s[i][1]*u;
    s[i][2] = s[i][1]  + s[i][2]*u;
    s[i][0] = C[4*i+1] + s[i][0]*u;
    s[i][1] = s[i][0]  + s[i][1]*u;
    s[i][0] = C[4*i]   + s[i][0]*u;
    s[i][2] += s[i][2];
  }
  r[0] = r[2] = r[5] = s[3][0];
  r[0] = s[2][0] + r[0]*u;
  r[2] = r[0]    + r[2]*u;
  r[5] = r[2]    + r[5]*u;
  r[0] = s[1][0] + r[0]*u;
  r[2] = r[0]    + r[2]*u;
  r[0] = s[0][0] + r[0]*u;
```

```
  r[5] += r[5];
  r[3] = s[3][2];
  r[3] += s[2][2] + r[3]*v;
  r[3] += s[1][2] + r[3]*v;
  r[3] += s[0][2] + r[3]*v;
  r[1] = r[4] = s[3][1];
  r[1] = s[2][1] + r[1]*v;
  r[4] = r[1]    + r[4]*v;
  r[1] = s[1][1] + r[1]*v;
  r[4] = r[1]    + r[4]*v;
  r[1] = s[0][1] + r[1]*v;
  return;
}


void EvalSurf ( double P[6][3], double u, double v, point *Cp ) {
  /* determine in which domain the parameter lies */
  int n = floor(min(-log2(u),-log2(v)))+1;
  double pow2 = pow2n[n-1]; /* Store pow2 as array of pow2n[i] = 2^i */
  u *= pow2;
  v *= pow2;
  if ( v < 0.5 ) {
    k=0;
    u=2*u-1; /* u += u-1; */
    v=2*v;   /* v += v; */
  }
  else if ( u < 0.5 ) {
    k=2;
    u=2*u;   /* u += u; */
    v=2*v-1; /* v += v-1; */
  }
  else {
    k=1;
    u=2*u-1; /* u += u-1; */
    v=2*v-1; /* v += v-1; */
  }
  /* Now evaluate the surface */
  P[0][0] = P[0][1] = P[0][2] = 0.0; /* Point on surface */
  P[1][0] = P[1][1] = P[1][2] = 0.0; /* p_u */
  P[2][0] = P[2][1] = P[2][2] = 0.0; /* p_v */
  P[3][0] = P[3][1] = P[3][2] = 0.0; /* p_uu */
  P[4][0] = P[4][1] = P[4][2] = 0.0; /* p_uv */
  P[5][0] = P[5][1] = P[5][2] = 0.0; /* p_vv */
  for ( i=0; i<eigennum[n]; i++ ) {
    /* Store pow(eigen.L[i],n-1) in two-dimensional array. eigenpow[][]
       Set int array of number of terms to use for each n. eigennum[]
       EvalSpline() doesn't know about n or k, so final scaling must
       be done in this function.
```

```
        First partial derivatives use pow2n[n] for 2^n.
        Second partial derivatives use pow4n[n] for 2^(2n).
    */
    double s[6];
    EvalSpline(eigen.x[i][k], u, v, s);
    double sf = eigenpow[i][n-1]*s[0];
    P[0][0] += sf*Cp[i][0];
    P[0][1] += sf*Cp[i][1];
    P[0][2] += sf*Cp[i][2];
    double sf2n = eigenpow[i][n-1]*pow2n[n]; /*Store?*/
    sf = sf2n*s[1];
    P[1][0] += sf*Cp[i][0];
    P[1][1] += sf*Cp[i][1];
    P[1][2] += sf*Cp[i][2];
    sf = sf2n*s[2];
    P[2][0] += sf*Cp[i][0];
    P[2][1] += sf*Cp[i][1];
    P[2][2] += sf*Cp[i][2];
    double sf4n = eigenpow[i][n-1]*pow4n[n]; /*Store?*/
    sf = sf4n*s[3];
    P[3][0] += sf*Cp[i][0];
    P[3][1] += sf*Cp[i][1];
    P[3][2] += sf*Cp[i][2];
    sf = sf4n*s[4];
    P[4][0] += sf*Cp[i][0];
    P[4][1] += sf*Cp[i][1];
    P[4][2] += sf*Cp[i][2];
    sf = sf4n*s[5];
    P[5][0] += sf*Cp[i][0];
    P[5][1] += sf*Cp[i][1];
    P[5][2] += sf*Cp[i][2];
  }
}
```

Data structures are required to navigate triangulated and quadded surfaces. We first consider what is necessary to navigate the triangulated surface of a convex hull in order to map the quadded sphere's topology onto the convex hull. Since the mapping is done only once and can be performed in at most a few seconds, no attempt will be made to optimize the process. We assume that the convex hull .ply file was exported from MeshLab in triangle format, including all vertices and unit normal vectors at the vertices. In addition the file contains all triangle face definitions, although they will not be used. Furthermore we assume that one of the Loop subdivision algorithms in MeshLab was used to obtain at least 100 to 1000 times more triangles than quads on the unit sphere. In essence, the triangulated convex hull file can be quite large, but this is not a problem because it used only once in a preprocessing step.

This much higher resolution file will not be further subdivided during the mapping process and only the vertices and their normal vectors will be used. As described earlier, a brute-force method is used to visit all convex hull vertex normal vectors $\underline{n}_h$ for each unit sphere normal vector $\underline{n}_s$ to and average the convex hull vertex vectors $\underline{v}_h$ for those $\underline{n}_h$ within a minimum angular distance of $\underline{n}_s$ to get the $\underline{v}_{hs}$ associated with the $\underline{n}_h = \underline{n}_s$. When the convex hull's surface subdivision is finer by an order of magnitude or more than the unit sphere's, the resulting vertices $\underline{v}_{hs}$ will more accurately coincide with the $\underline{n}_h = \underline{n}_s$ assigned from the unit sphere.

The data structures for the quadded convex hull, which will have the same topology as the quadded unit sphere used to convert it, will be required for vertices, edges, and faces. Each vertex data structure must contain its valence number (ordinary or extraordinary), a set of pointers to its common edges, and a set of pointers to its common faces. Each edge data structure must indicate whether or not it contains an extraordinary vertex, and it must point to its two common vertices and two common faces. Each face data structure must indicate whether or not it contains an extraordinary vertex, and it must point to its four common vertices and four common faces.

Each ordinary face will have associated with it bicubic B-spline data sufficient to evaluate any point and partial derivatives on its surface. In addition, when either of its u or v variables goes out of bounds, the face data structure will contain the necessary information to smoothly transition across an edge boundary to an adjacent face data structure and continue from there. The transition may be to another ordinary face or it may be to an extraordinary face. The transitions will be the same because the ordinary and extraordinary face interfaces are identical.

Each extraordinary face will also have associated with it substantially more bicubic B-spline data sufficient to evaluate any point and partial derivatives on its surface. External to the special functions used to compute points and partial derivatives on an extraordinary face, the u and v variables are subject to the same bounds as the ordinary faces, but internal to the special functions the u and v variables may be scaled numerous times to obtain the final results. The bottom line is that when a u or v variable associated with an extraordinary face goes outside its limits the transition will be to an adjacent ordinary or extraordinary face. The only requirement is that each face must have pointers to adjacent faces so the transition can be made smoothly.

Most of the setup can be made in a preprocessing step once and stored in a file for follow-on use. The setup can start by adding data to each face structure. Faces on the unit sphere and each convex hull are generated starting at the top (most positive z coordinates) in counterclockwise rings around the positive z-axis and progressing downward to the most negative z coordinates. Each ordinary face requires 16 projected vertices arranged in a four by four grid around the face so that the first

projected vertex is in the lower-left corner, the last projected vertex is in the upper-right corner, and they are arranged in rows, left to right, bottom to top. In this manner the u-axis aligns with the rows of projected vertices and the v-axis points upward toward the top row.

The u and v coordinate axes on each face are oriented relative to adjacent faces as much as possible so that transitions across adjacent edges will be consistent. Thus wherever possible the u-axes are aligned end to end, as are the v-axes. The primary difference between the set of projected vertices for an ordinary face versus an extraordinary face associated with a valence three vertex is that each ordinary face has a ring of 12 vertices and eight faces surrounding it, but each extraordinary face has a ring of only 10 vertices and seven faces surrounding it. Thus each ordinary face will have an array of 16 projected vertices, and each extraordinary face will have an array of 14 projected vertices.

It is helpful but not necessary to arrange the u and v coordinate axes and the 16 or 14 projected vertices associated with each face to increase the number of aligned axes. The vertex-numbering scheme for ordinary faces can be made consistent as discussed above, where the origin of each center face is at the sixth projected vertex with linear index 5. The situation is different for extraordinary faces because their sets of 14 projected vertices are not symmetric. Three extraordinary faces surround each extraordinary vertex, and each of the three faces must be assigned a different arrangement of projected vertices consistent with placing origins of the u and v coordinate axes on the extraordinary vertex as required by Stam's algorithm.

Not all faces can be joined to their neighbor faces such that each u and v variable is a continuation of the respective adjacent u and v variable. Faces in the quad strip rings around the top and bottom major quads make four 90° turns, and only at these turns is where continuations of u and v variables cannot be obtained. When n is even the total number of 90° turns on each major quad is $2n$, so a total of $4n$ faces will not have continuation of the u and v variables across adjoining edges within each major quad. A similar number exists for when n is odd. The extraordinary vertices and faces are also on the corners.

Transitions between two adjacent faces will be called regular when their local coordinate frames have the same orientation. Let u_r^+ and v_r^+ represent ordinary positive transitions from one face to the next where u or v has gone beyond +1. In these cases the transition equations are ($u = u - 1$ and $v = v$) or ($u = u$ and $v = v - 1$). Similarly let u_r^- and v_r^- represent regular negative transitions from one face to the next where u or v has gone beyond –1. In these cases the transition equations are ($u = u + 2$ and $v = v$) or ($u = u$ and $v = v + 2$).

The minor faces are laid out in counterclockwise rings around the positive z-axis on the face of the top quad 0. Each ring is divided into four segments where each segment lies along one side of a "square." When n is even, the number of minor

faces in each segment of the ith ring is $2i-1$, and when n is odd, the number of minor faces in each segment is $2i$. This last equation shows that when n is odd and $i=1$ there are eight minor faces in the first ring surrounding a single face centered on the positive z-axis. Thus when n is odd the number of minor faces in each edge of the first ring is two, whereas it is one when n is even. Likewise when n is odd the number of faces in each edge is even, and when even the number of faces is odd.

Each successive segment of minor faces in a ring is essentially a version of the previous segment rotated 90° counterclockwise around the positive z-axis. Thus the u-v coordinate axes of each successive segment of minor quads will be rotated 90° counterclockwise relative to the previous segment. As each successive ring expands outward, all coordinate axes of each expanding set of adjacent ring segments will have the same orientation. Thus the major quad 0 may be split roughly along its two major diagonals into four identical triangular segments of minor quads, each rotated 90° counterclockwise relative to the previous one. When n is odd the u-v coordinate axes of the lone minor face centered on the positive z-axis are aligned with those in the first triangle of minor quads.

A simple set of rules does not exist to transition the u and v variables across the four edges of each minor quad. However there is a small set of rules representing all transitions, and a preprocessor can determine which ones apply to each minor quad's four edges. The majority of transitions are regular, where the adjacent faces have the same orientation and the respective u and v axes are aligned. The remaining transitions are irregular, where adjacent faces are rotated relative to each other by 90°, 180°, or 270°. These relative rotation angles between the u and v coordinate axes of adjacent faces will determine the transition equations to be applied there.

Let us consider transitions between adjacent faces when the relative angles are 0°, 90°, 180°, and 270°. Assume that transitions take place from the primary face to the secondary face, which has been rotated by one of the four angles.

Starting with the secondary face rotated 0° assume first it is on the positive u side of the primary face. Let the u-axis point to the right and the v-axis point upward in the primary face. Then the u-axis points to the right and the v-axis points upward in the secondary face. The four possible transitions from the primary to secondary face are

1. $u=u-1$ and $v=v$ when the secondary face is on the positive u side.
2. $u=u$ and $v=v-1$ when the secondary face is on the positive v side.
3. $u=1+u$ and $v=v$ when the secondary face is on the negative u side.
4. $u=u$ and $v=1+v$ when the secondary face is on the negative v side.

With the secondary face rotated 90° assume first it is on the positive u side of the primary face. Let the u-axis point to the right and the v-axis point upward in the primary face. Then the u-axis points upward and the v-axis points to the left in the

rotated secondary face. The four possible transitions from the primary to secondary face are

5. $u = v$ and $v = 2 - u$ when the secondary face is on the positive u side.
6. $u = v - 1$ and $v = 1 - u$ when the secondary face is on the positive v side.
7. $u = v$ and $v = -u$ when the secondary face is on the negative u side.
8. $u = 1 + v$ and $v = 1 - u$ when the secondary face is on the negative v side.

With the secondary face rotated 180° assume first it is on the positive u side of the primary face. Let the u-axis point to the right and the v-axis point upward in the primary face. Then the u-axis points to the left and the v-axis downward in the rotated secondary face. The four possible transitions from the primary to secondary face are

1. $u = 2 - u$ and $v = 1 - v$ when the secondary face is on the positive u side.
2. $u = 1 - u$ and $v = 2 - v$ when the secondary face is on the positive v side.
3. $u = -u$ and $v = 1 - v$ when the secondary face is on the negative u side.
4. $u = 1 - u$ and $v = 1 - v$ when the secondary face is on the negative v side.

Finally with the secondary face rotated 270° assume first it is on the positive u side of the primary face. Let the u-axis point to the right and the v-axis point upward in the primary face. Then the u-axis points downward and the v-axis to the right in the rotated secondary face. The four possible transitions from the primary to secondary face are

1. $u = 1 - v$ and $v = u - 1$ when the secondary face is on the positive u side.
2. $u = 2 - v$ and $v = u$ when the secondary face is on the positive v side.
3. $u = 1 - v$ and $v = 1 + u$ when the secondary face is on the negative u side.
4. $u = -v$ and $v = u$ when the secondary face is on the negative v side.

The next step is to determine orientations of the four minor quads adjacent to the current minor quad whose rules are being established. The rules can be established in an orderly manner, starting at the center of quad 0 and working downward to the center of quad 6.

When n is odd the first minor quad in major quad 0 is centered on the positive z-axis, and the last minor quad in major quad 6 is centered on the negative z-axis. The u and v-axes of the first minor quad are aligned with the respective x and y-axes of the surface. When n is even there is no first or last minor quad centered on the z-axis, but a first ring of four minor quads centered on the positive z-axis and a last ring of four minor quads centered on the negative z-axis. When n is even the u and v-axes of the first minor quad are aligned with the respective x and y-axes of the surface. Whether n is even or odd, rings of minor quads expand outward from the first minor quad or ring in counterclockwise order around the positive z-axis, providing an orderly way of setting the transition rules, based on the known relationships of each ring's minor quads to their adjacent quads.

The relationships between adjacent minor quads are somewhat involved but were established when the faces were generated. The following will first describe the setup for the single minor quad centered on the positive z-axis. Then it will describe the setup for the four minor quads centered on the positive z-axis. Next it will describe the setup for the eight minor quads in the first ring surrounding the single minor quad centered on the positive z-axis. And finally it will describe the setup for the 12 minor quads in the second ring surrounding the first ring of four quads centered on the z-axis. The resulting rules will apply to all remaining minor quads in the first major quad 0.

Whether n is even or odd the first minor quad establishes the orientation of all $6n^2$ minor quads on the surface. As noted above its local u and v-axes are associated with the object's x and y-axes. If the first minor quad is centered on the positive z-axis the second minor quad will be placed on its positive u and v vertex corner and with the same u and v-axis alignment. These two minor quads will have no edges in common, so no rules are generated there. The third minor quad will be placed on the negative u-axis side of the second minor quad, and will appear on the positive v-axis side of the first minor quad.

The second and third minor quads have completed the first side of the first ring, and the fourth minor quad at the next corner must turn 90° to become the first quad in the second side. This minor quad is placed on the negative u-axis side of the second minor quad, and since it occupies a corner position in the ring, it has no edge in common with the first minor quad. The fifth quad is placed next to the fourth quad in the same manner as the third quad was placed next to the second quad. This quad will have an edge in common with the first quad. These two quads complete the second side of the ring. In a similar manner quads six and seven will round out the third edge and quads eight and nine the fourth edge to complete the ring. Thus quads 3, 5, 7, and 9 will have edges in common with the first quad.

The four transition rules between quad 1 and quads 3, 5, 7, and 9 can be established, where quad 3 is on the +v-axis side, quad 5 is on the –u-axis side, quad 7 is on the –v-axis side, and quad 9 is on the +u-axis side of quad 1. The important observations to make when assigning the transition rules is that quad 3 has been rotated by 0°, quad 5 by 90°, quad 7 by 180°, and quad 9 by 270° relative to quad 1. Thus the above sets of rules can be used to determine which apply here.

With these four transitions set for quad 1, the opposite four transitions for quads 3, 5, 7, and 9 back to quad 1 can be set. This completes the transitions for quad 1, and two additional transitions for quads 2 through 9 are easily set up by their relative positions and orientations in the ring.

Starting with quad 2 we know that quads 2 and 3 have the same orientation and that quad 3 is on the –u-axis side of quad 2. Likewise quad 4 is on the –u-axis side of

quad 3 but it has been rotated 90°, so one of the above 90° rules apply. Then quad 5 is on the –u-axis side of quad 4 but with the same orientation as quad 5. In a similar manner quad 6 is on the –u-axis side of quad 5 and rotated 90° relative to quad 5, so the same 90° rule applies, and quad 7 follows quad 6 with the same orientation as quad 6. Since rotations and translations in the ring are all relative it is possible to set up the same four transitions for each quad pair, 2&3, 4&5, 6&7, and 8&9. Now there remains one transition rule to set for each of the side quads 3, 5, 7, and 9, and two transition rules to set for each of the corner quads 2, 4, 6, and 8.

Before describing how to set up these remaining transitions we consider the case of the single first ring of four quads when n is even. The first quad is set up in the positive x and y quadrant with its u and v-axes oriented with the respective x and y-axes. In this case all four quads lie on corners of the ring, so each successive quad is rotated 90° relative to the previous one, and like above each next quad is placed on the –u-axis side of the quad. Thus two transition rules of these four corner quads are established, leaving two transition rules remaining for each quad.

At this point a similar set of transition rules can be established for each successive ring of minor quads whether n is even or odd. In each new ring the first minor quad is set up in the positive x and y quadrant just off the positive u and v corner of the first quad of the previous ring, and its u and v-axes are oriented with the x and y axes. This corner quad starts the first ring edge segment and is not adjacent to any quad in the next inner ring. Then one or more side quads will be added sequentially along the first edge, each off the –u-axis and with the same orientation as the corner quad. Next a second corner quad, rotated 90°, is added on the –u-axis side of the last side quad to start the second edge of the ring. Then one or more side quads are added to finish out the second edge. The third and fourth edges of the ring are set up in a similar manner.

Each of the added side quads is adjacent to a quad in the next inner ring and will be used to set one transition rule between adjacent quads in the inner and outer rings. The adjacent quads in the inner ring are on the -v-axis side of the quads in the outer ring, and most of them have the same orientation. The exception is that each corner quad in the inner ring is adjacent to two quads in the outer ring, so each corner quad is aligned with one side quad and rotated 90° relative to the other. Once these rules have been set for the quads in the inner ring, they each have four transition rules and are complete, and all side quads in the outer ring each have one transition rule set.

Each quad in the new outer ring is adjacent to two quads, and the previously described formulas for setting up two additional transition rules for each quad can be applied to give each side a total of three transitions and each corner quad two transitions. At this point the outer ring becomes the next inner ring and the next outer ring is ready to be added. The process continues until the last outer ring of the first major quad 0 is completed and becomes the first inner ring of minor quad

strips to be wrapped sequentially across faces of the major quads 2, 3, 4, and 1 in that order.

Setting up the transition rules for the $4n^2$ minor quads comprising major quads 2, 3, 4, and 1 is simple because all adjacent quads have the same orientation in terms of their u and v coordinate axes. These u and v-axes axes don't have predictable orientations relative to the x and y-axes, but their orientations relative to each other when the sides are rolled out flat are predictable. Thus the set of four transitions for the 0° relative orientation are applied with the proper quad indices. Once the transitions for these $4n^2$ minor quads have been set, the last outer ring of minor quads becomes the first outer ring of the remaining n^2 minor quads in the last major quad 5.

Since the minor quad rings in quad 5 are progressing inward toward its center, each new ring is called the inner ring, and each previous inner ring become the next outer ring. The last outer ring of quads from the previous four major quads, which becomes the first outer ring of quads for quad 5, has all four transition rules set for all minor quads. Thus this ring is used to set some transition rules for minor quads in the first inner ring.

Each minor quad ring in major quad 5 has four corners and is divided into four edge segments, as was done for the rings in major quad 0. The minor quads are still arranged counterclockwise around the positive z-axis, but they appear to be arranged clockwise when viewed from the negative z-axis direction outside the convex hull.

The main differences between the quad rings on major quad 5 versus major quad 0 is that inner quad rings are being generated on major quad 5 as opposed to the outer quad rings on major quad 0. In addition the face definition vertex ordering on major quad 5 is reversed from major quad 0, so the face normals will be pointing outward. However, the first minor corner quad of the first edge segment is still placed in the positive x, y quadrant of quad 5, and the adjacent minor quads are appended on the –u-axis edge of each quad. And each new minor quad is rotated 90° relative to the last quad to start the next edge segment.

Thus when setting up the four transition rules for the new minor quads the first rules are obtained from the adjacent minor quads on the previous outer ring of quads. Since the new inner ring of quads is inside the outer ring, each side quad will be adjacent to one outer quad and generate one rule, and each corner quad will be adjacent to two outer quads and generate two rules. Then since each minor quad is adjacent to two other minor quads in the new inner ring, two additional transition rules are generated for each one. This gives a total of four rules for each corner quad, which completes their transition rules. And each side quad in the inner ring now has three transition rules. Then the inner ring becomes the new outer ring for generating the next inner ring.

Transition rules for the four minor corner quads in the last ring, when n is even are easily obtained from the previous outer ring as are the transition rules for the last single minor quad when n is odd.

With four transition rules set and stored with each quad, the Newton-Raphson iteration algorithm can move the two minor quad variables in any direction on each convex hull without interruption. When transitioning between minor quads sometimes the u and v variables and/or their directions may be reversed according to their stored rules. Before the next Newton-Raphson iteration the variables are adjusted to be consistent with the new minor quads.

In order to approximate each surface associated with one of the $6n^2$ minor quads a 4 by 4 array of 16 control vertices on and around each ordinary quad must be identified, and a distorted 14-vertex array of control vertices on and around each extraordinary quad must be identified. Twenty-four of the minor quads are extraordinary. Each ordinary quad has a u-v coordinate system with its origin in one corner. The 16 control vertices are numbered locally with vertex 0 located in the –u, –v quadrant and numbered in rows parallel to the u-axis in increasing order in the positive u direction. Thus the first row of control vertices, numbered from 0 to 3, is on the negative v side of the u-axis. The second row of control vertices, numbered from 4 to 7, lies along the u-axis. The third and fourth rows are on the positive v side of the u-axis.

A face is ordinary when its four vertices each have a valence of four and are themselves ordinary. An ordinary face's 16 support vertices consist of the four vertices at its corners and the three additional vertices common to each of the faces at its four corners. It is then a matter of pulling vertex indices from the right faces in the right order to assemble them into the 4 by 4 arrays.

When a face is extraordinary, Stam's algorithm requires that the origin of the u-v coordinate system be located at the extraordinary vertex. The transition rules for an extraordinary face and faces adjacent to it must be adjusted if the origin of its assigned u-v coordinate system is not at the extraordinary point. There are eight extraordinary vertices and nine edges common to the three extraordinary faces surrounding each extraordinary vertex. Thus up to 72 transition rules may need to be adjusted to account for relocating the u-v coordinate systems. It turns out that one third of the coordinate systems are located correctly, so 18 must be relocated and 48 transition rules adjusted. Making these few changes will be much easier than changing how coordinate systems are assigned to the majority of faces.

The set of $6n^2+2$ vertices on the unit sphere and on any convex hull whose topology is determined by that unit sphere is stored exactly in ring-by-ring order, starting at the center of major quad 0 and ending at the center of major quad 5. The set of $6n^2$ minor quad faces on the unit sphere and on any convex hull whose

topology is determined by that unit sphere is stored exactly in quad-strip-ring by quad-strip-ring order, starting at the center of major quad 0 and ending at the center of major quad 5. The eight extraordinary vertices are located at the eight common corners of the six major quad faces, the same as would be for the six faces of a cube. And four extraordinary minor quad faces are located inside each major quad, adjacent to the four extraordinary vertices at the corners. Thus three extraordinary faces surround each extraordinary vertex. Corner vertices and minor quad faces of the major quads are easily detected, and special u-v coordinate frames and transition rules can be set up as they are generated. Using this approach the entire setup process need be performed only once on the unit sphere, and the results will apply to any convex hull whose topology is based on that unit sphere.

The most difficult part is setting up the u-v coordinate systems on the 24 extraordinary minor quad faces with origins at the extraordinary vertices as required by Stam's algorithm. The basic requirement is that each $\vec{u} \times \vec{v}$ points outward, as it must for all $6n^2$ minor quad faces. Thus the u-axes point in a counterclockwise direction and the v-axes in a clockwise direction along the edges of the major quads as viewed from the outer faces.

Stam's algorithm requires a specific numbering scheme for the 14 vertices on and adjacent to each extraordinary face, which is unrelated to the numbering scheme used for the 16 vertices on and adjacent to each ordinary face. Each extraordinary face lies inside one corner of a major quad and is surrounded by seven ordinary faces. Three ordinary faces are in the major quad containing the extraordinary face, and two ordinary faces each are in the other two major quads. The local numbering scheme of the 14 vertices is mostly clockwise around the extraordinary vertex, which is first with local index of 1. The actual local vertex indexing order is best illustrated using a figure because it has no simple explanation.

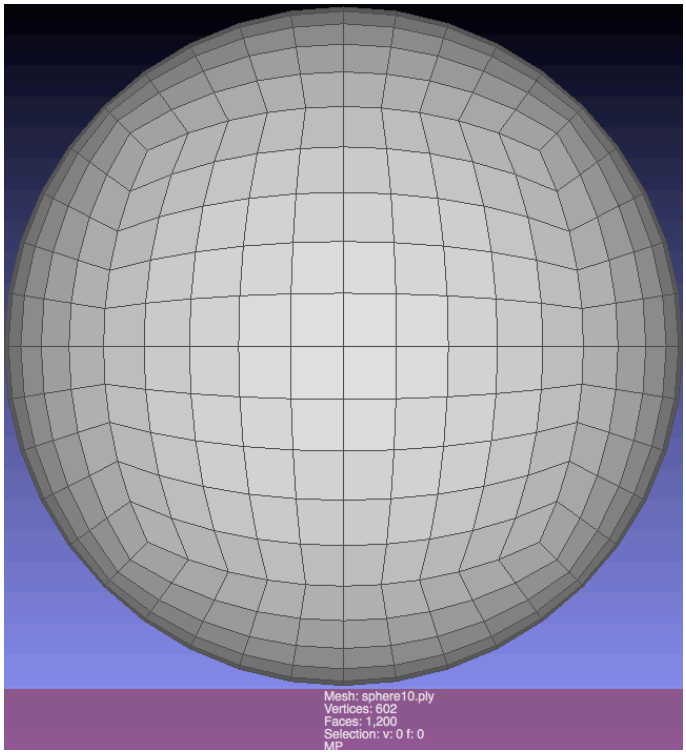
Whether a given face is extraordinary or not, setting up transition rules between it and its four adjacent faces as they are being generated will be easier because the assignment of their u-v coordinate systems will be known at the time. It is simply a matter of picking from among the four sets of transition rules presented earlier, as they cover all possible combinations of relative u-v coordinate frame positions and orientations on adjacent faces.

One extraordinary vertex occurs at each corner in the edge ring of vertices in major quad 0, and one extraordinary face occurs at each corner in the outer ring of minor quad faces in major quad 0. The first ring of minor quad faces surrounding major quad 0 and passing along one edge of each major quad 1 through 4 contains two extraordinary faces adjacent to each extraordinary face in major quad 0. This accounts for four of eight extraordinary vertices and 12 of 24 extraordinary faces.

One extraordinary vertex occurs at each corner in the edge ring of vertices in major quad 5, and one extraordinary face occurs at each corner in the outer ring of minor

quad faces in major quad 5. The last ring of minor quad faces surrounding major quad 5 and passing along one edge of each major quad 1 through 4 contains two extraordinary faces adjacent to each extraordinary face in major quad 5. This accounts for the other four of eight extraordinary vertices and the other 12 of 24 extraordinary faces.

Thus setting up the special transition rules for extraordinary vertices and faces occurs in and around the rings of vertices and faces at the edges of quads 0 and 5. In addition the special transition rules are set up at and immediately surrounding the four corners of these two quads where the extraordinary vertices and faces are located. Thus it is a simple matter of detecting these small regions out of the entire $6n^2+2$ vertices and $6n^2$ faces and taking special actions there. The primary requirement as noted earlier is to reorient 16 of the 24 u-v coordinate systems on extraordinary faces, which then requires different sets of transition rules from the norm.



The above image shows one major quad of a unit sphere with $6n^2+2=6*10^2+2=602$ vertices and 600 minor quad faces. Four corners of this major quad and two corners of four adjacent major quads are visible. The major quad corners are at the four extraordinary vertices, which have a valence of three, and the 12 extraordinary faces each contain one extraordinary vertex. If this major quad is taken as quad 0, then its four edges pass through the extraordinary vertices. In addition, it can be seen that the outermost ring of minor quad faces inside quad 0 contains four of the 12 extraordinary faces. The remaining eight extraordinary faces

lie in the ring of quad faces just outside quad 0. And it can be seen that three extraordinary quad faces are clustered around each extraordinary vertex at corners of the major quads.

If this view is taken as major quad 0 looking down from the positive z-axis, the vertices and minor quad faces are generated counterclockwise in rings extending outward from the quad's center outward to its edges. Then additional rings of vertices and minor quad faces are expanded outward, parallel to the edges of quad 0, and downward across the four major side quads 1 to 4.

Additional counterclockwise rings of vertices and minor quad faces continue expanding downward across the four major side quads until reaching the edges of major quad 5 on the bottom. Imagine viewing major quad 5 from inside the sphere, still from the positive z-axis direction, so the view is just like the above image. The last ring of minor quads just outside quad 5 contains eight additional extraordinary faces, each with one extraordinary vertex. The four extraordinary vertices are at the corners of quad 5 and at two corners each of quads 1 to 4. Just inside quad 5 is another ring of minor quad faces containing the last four extraordinary faces. The processes for generating the extraordinary vertices and faces near the edges of quad 5 are nearly identical to those near the edges of quad 0. The primary difference is that the u-v coordinate frames are not assigned to the two sets of faces in the same way, so setting the transition rules for the extraordinary faces near quad 5 are somewhat different than for the similar extraordinary faces near quad 0.

An alternative method of assigning u-v coordinate systems to the $6n^2$ minor quad faces will reduce the complexity of setting up transitions across their edges. All minor quad u-v coordinate axes within each major quad will be given the same orientation, except that in three extraordinary faces of each major quad they will be rotated by 90°, 180°, and 270° respectively to place their origins at extraordinary points required by Stam's algorithm. Thus only 18 of the $6n^2$ u-v coordinate systems will be rotated, and all rotations will take place in extraordinary faces.

Within each major quad, transitions between all adjacent ordinary minor quads will use the 0° rules. Only transitions between adjacent ordinary faces and extraordinary faces whose u-v axes have been rotated will use the 90°, 180°, or 270° rules as necessary. All remaining transitions between minor quad faces will be across edges between adjacent major quads. Roughly one half of these transitions will use the 0° rule and the remaining half will split roughly between the 90°, 180°, and 270° rules. The set of transition rules applied to the minor quad faces along each pair of adjacent major quad edges will be the same except at the edge ends where the extraordinary vertices and faces are located. In these few locations it will be straightforward to select alternate 0°, 90°, 180°, or 270° transition rules as necessary.